Tipe carnet de bord explication

1 - Recherche Biblio

La bibliothèque OpenCV est une bibliothèque de traitement d'image avec des fonctions de seuillage déjà intégré, de nombreuses fonctionnalités comme l'approximation de contours de forme sur une image et les filtres de seuillage les plus connus. Cette bibliothèque permet de simplifier l'acquisition d'un flux vidéo et le traitement des informations de chaque image.

Recherche d'outil pour l'analyse d'image

Dans un premier temps je me suis renseigné sur les bibliothèques python permettant l'analyse d'image car c'est le langage informatique que je maitrise le mieux.

Je me suis tourné vers la bibliothèque OpenCV qui est réputée dans ce domaine.
Il m'a donc fallu récupérer le flux vidéo de ma caméra.

Etant donné que je travaille sur un pc portable, j'ai pu travailler directement sur le flux des caméras dont il est muni. Les images obtenues grâce à ces caméras sont de bonne qualité.

Programme récupération du flux vidéo

OpenCV permet de capturer le flux vidéo d'une caméra grâce à la classe « VideoCapture ».
Grâce à la fonction "read()" je peux récupérer l'image capturé à l'instant actuel par ma caméra.
Il m'a donc suffi de faire une boucle while avec une condition de sortie étant l'appui d'une touche (la touche 'echap' dans mon cas) et de récupérer à chaque début de boucle l'image capturée par la caméra.

Pour analyser mes images, j'affiche le flux vidéo

Programme de détection d'objet par couleur

J'ai essayé dans un premier temps de faire la détection de contours d'un objet unicolor.

Pour cela, j'ai récupéré la valeur de chaque pixel de l'image, il s'agit d'un triplet contenant les valeurs d'intensité de rouge, vert et de bleu (R, G, B).
Ensuite il m'a fallu appliquer un filtre sur cette liste de valeur.
Le filtre est le suivant :

si chacune des valeurs d'intensité du pixel est comprise entre le minimum autorisé et le maximum. Donc si (R_min, G_min, B_min) < (R, G, B) < (R_max, G_max, B_max)

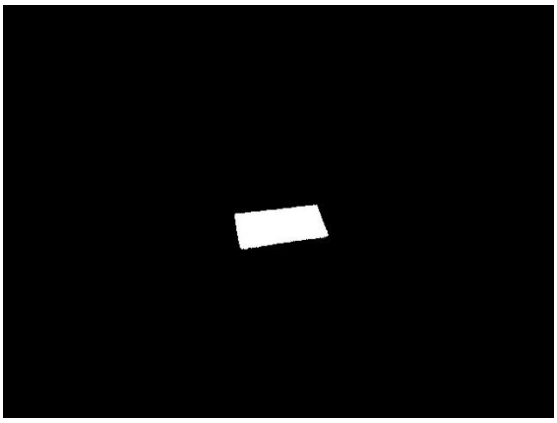
alors le pixel devient blanc

sinon

    le pixel devient noir

exemple :

->



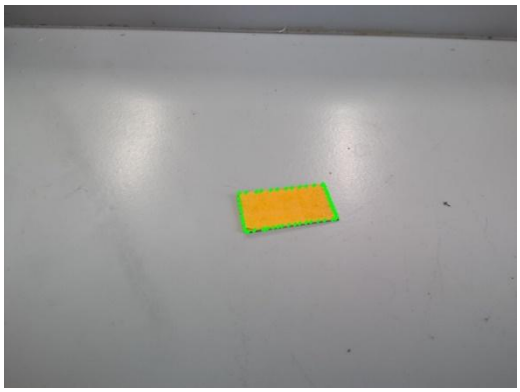
Ensuite j'ai utilisé la fonction "findContours" d'OpenCV, qui permet la détection des contours de forme.

Si j'avais dû coder cette fonction moi-même j'aurais regardé les voisins de chaque pixel qui n'est pas au bord de l'écran et pour chacun d'entre eux j'aurai utilisé les conditions suivantes pour savoir si c'est un contour d'objet :

- si le pixel est blanc :
    - s'il est entouré les pixels en haut, en bas, à gauche et à droite sont blanc aussi :
        - alors ce n'est pas un contour
    - sinon :
        - # c'est un contour
        - on ajoute ses coordonnées à la liste de coordonnées de contours

Pour avoir le code le plus optimisé j'ai utilisé la fonction d'OpenCV. Elle renvoie la liste de coordonnées des points permettant de définir les contours de mon objet.

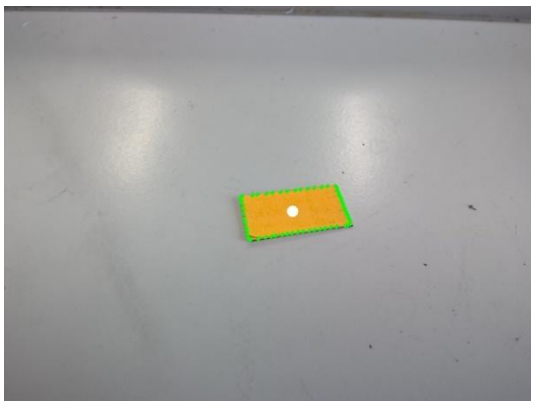
Voici ce que ça donne ajoutant ces points sur l'image



Ensuite il me faut trouver le centre de l'objet pour pouvoir savoir s'il est contenu dans la zone d'acceptance (la zone où l'on veut centrer notre objet)

Pour cela j'aurais pu faire la moyenne des coordonnées des points des contours de l'objet. Mais OpenCV permet de trouver le centre de ces points grâce aux moments spatiaux (aussi appelés moments bruts) grâce à un calcul assez simple.
J'ai donc privilégié la méthode des moments pour une plus grande lisibilité du programme.

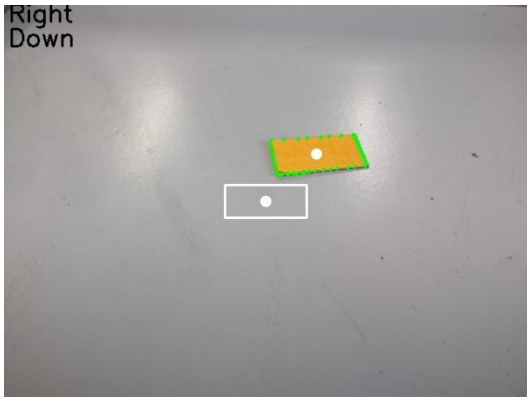
On a donc maintenant le centre de notre objet.

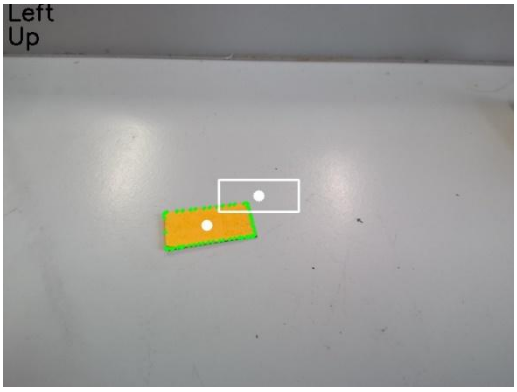


Programme ordre de déplacement de la caméra

Notre but étant de centrer l'objet sur l'image, j'ai décidé de créer une zone d'acceptance rectangulaire au centre de l'image.

Si le centre de notre objet n'est pas dans cette zone d'acceptance, on calcule la différence de pixel entre le centre de l'image et le centre de l'objet et on donne l'ordre de déplacement de la caméra proportionnellement à cette distance.





Ce programme a comme avantage de ne pas être affecté par le background pour détecter un objet mais reste sensible à la luminosité et l'éclairage de l'objet. Il ne peut détecter que des objets unicolores et la couleur doit être choisie au lancement du programme.

Les valeurs du filtre de couleur ont été trouvé expérimentalement en modifiant petit à petit les valeurs d'acceptance du filtre, et ce pour chacune des couleurs.

Les valeurs sont ensuite stockées dans le programme.

Détection d'objet par contraste

Les différences entre ces programmes sont sur les filtres.
La détection de contours, le calcul du centre de l'objet, le positionnement au centre de l'image est géré exactement de la même manière.

Le but de cette seconde version est de pouvoir détecter des objets de différentes couleurs sans relancer le programme et de pouvoir détecter des objets qui ne sont pas unicolores.

Le filtre de ce nouveau programme est le suivant :

D'abord on transforme l'image de couleur obtenue par la caméra en nuance de gris puis on applique un filtre gaussien. Et enfin on filtre les en fonction des valeurs de pixel comme pour le premier programme.

Grey = 0.299 · Red + 0.587 · Green + 0.114 · Blue.

C'est la formule de la norme NTSC, cette norme est le standard international de codage couleur du signal vidéo analogique

Ce filtre nécessite un fond à peu près uniforme, un éclairage assez clair pour fonctionner correctement. Il est plus polyvalent pour la détection d'objet mais beaucoup plus sensible au background de l'image.

Pour être sûr de bien comprendre comment fonctionnait mon programme, j'ai voulu recoder à la main les fonctions de OpenCV. Mon algorithme peut se décomposer en 4 étapes clés : la capture de l'image sous forme de tableau de valeurs ; la conversion de l'image en image de gris ; la mise en valeur de l'objet et enfin la détection des contours de celui-ci.

J'ai donc récupéré le flux vidéo de la caméra puis pour chaque image je fais le traitement suivant : Je récupère la valeur du pixel qui est un tuple avec l'intensités de rouge vert et bleu et j'applique la formule de la norme internationale NTSC pour obtenir le niveau de gris équivalent.
Grey = 0.299 · Red + 0.587 · Green + 0.114 · Blue.

Je sélectionne ensuite les valeurs comprises dans un interval choisi pour le récupérer que les zones plus foncées. Les pixels des zones choisies sont transformés en blancs et les autres en noirs.

Pour détecter les contours, je regarder pour chacun des pixels de l'image qui ne sont pas sur les bords s'il est blanc. Si c'est le cas, ça veut dire qu'il appartient à l'objet qu'on cherche donc je regarder si les pixels en haut, en bas, à droite et à gauche sont blanc aussi. Si c'est le cas, il est dans l'intérieur de l'objet donc c'est n'est pas un contour. Si l'un des pixels est noirs alors il s'agit d'un contour. Je stocke donc des coordonnées dans la liste des contours. Puis je fais la moyenne des coordonnées des contours pour avoir le centre de l'objet.

La complexité de mon programme est en $O(n^2)$ mais je n'ai pas trouvé comment faire pour la réduire plus.
En faisant tourner mon programme, les calculs prennait trop de temps à être effectuer donc le programme ne tournait pas correctement et ne pouvait réagir assez rapidement. J'ai donc décider de reprendre mon programme utilisant les fonctionnalités d'OpenCV qui est plus rapide.

Il était temps maintenant de voir si je pouvais utiliser ces programmes pour un suivi d'objet en temps réel. J'ai d'abord pensé à construire mon propre slider mais après des discussions avec mes professeurs, cela avait un trop gros coût et m'ont conseillé de trouver des alternatives.
J'ai trouvé dans le labo un slider éducatif, je me suis donc penché sur les possibilités d'utilisations que je pouvais en faire

J'ai fait face à plusieurs problèmes comme l'absence de donnée sur le fonctionnement du slider (aucune documentation en ligne sur le fonctionnement sur son commandement par python). J'ai donc contacter l'entreprise qui a fourni le slider à mon lycée et j'ai pu obtenir la documentation complète de celui-ci.

Le problème devenait le fait que le slider doive faire une prise d'origine entre chaque ordre de translation. Je pensais que c'était inutilisable. J'ai voulu donc construire le propre slider mais faisant face à de nombreuses difficultés et car la fabrication d'un slider nécessite du matériel couteux. Je me suis repenché sur le slider du labo.

J'ai remarqué que le slider lisait les ordres et que chaque nouvel ordre effaçait de sa mémoire les précédents et que seul le dernier ordre se faisait. J'ai donc cherché à utiliser ce fonctionnement pour contourner ce problème, j'ai donc fait en sorte que chaque nouvel ordre de translation soit lu par le programme juste avant la fin de la translation en cours, ce faisant le slider considère que la translation n'est pas finie et continue son mouvement.

Après avoir réussi à avoir un suivi d'objet en temps réel avec le filtre par couleur, j'ai essayé de fluidifier les transitions de mouvement du slider (en vain). Je me suis penché sur d'autres filtres, j'ai essayé le filtre par contraste dans le labo. Cependant celui-ci ne pouvant pas différencier les objets en mouvement dans autres dans un environnement complexe.

Le filtre de Canny est un filtre qui permet de mettre en valeur les contours, rapide et très fiable. Son fonctionnement étant le suivant : on converti l'image en gris pour n'avoir qu'une valeur par pixel.

Prétraitement avec un flou gaussien

L'image est d'abord lissée avec une convolution gaussienne pour réduire le bruit.

Calcul du gradient d'intensité

Ensuite on calcule l'approximation du gradient de l'image grâce à des matrices de convolution (filtres de Sobbel).

Suppression des non-maxima

On affine la détection des bords en supprimant les pixels qui ne correspondent pas à des maximas locaux du gradient dans la direction de θ. Chaque pixel est comparé à ses voisins alignés avec le gradient :

- Si un pixel n'est pas un maximum local, il est supprimé.

Cette étape affine les contours pour éviter un épaississement excessif.

Seuillage par hystérésis

Deux seuils T_{haut} et T_{bas} sont définis en fonction des valeurs des maximas pour avoir un filtre adaptatif :

- Les pixels avec un gradient supérieur à T_{haut} sont conservés comme contours.
- Ceux entre T_{haut} et T_{bas} sont gardés seulement s'ils sont connectés à un pixel déjà détecté.
- Les pixels en dessous de T_{bas} sont éliminés.

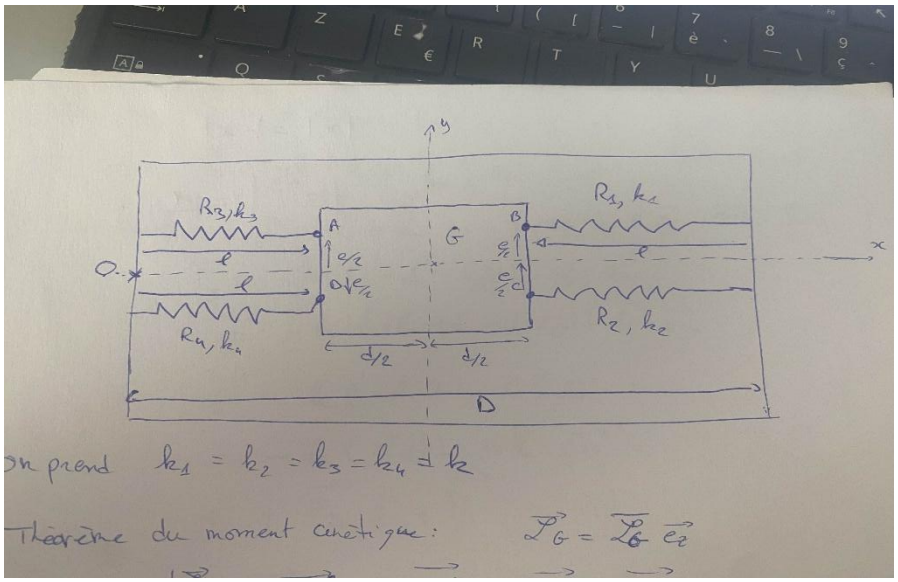
Après l'avoir implémenté à la main pour être sûr de bien comprendre son fonctionnement et de comprendre l'utilité de chacune des étapes. Je vais me pencher sur la méthode par soustraction de fonds.

Tentative de résolution d'à-coups lors du suivi :

Problème : la caméra est beaucoup secouée lors du suivi, le mouvement n'est pas fluide et ressemble plutôt à une succession de créneaux pour la vitesse

Idée première : Construire un stabilisateur passif

Utilisation de ressort avec frottements :



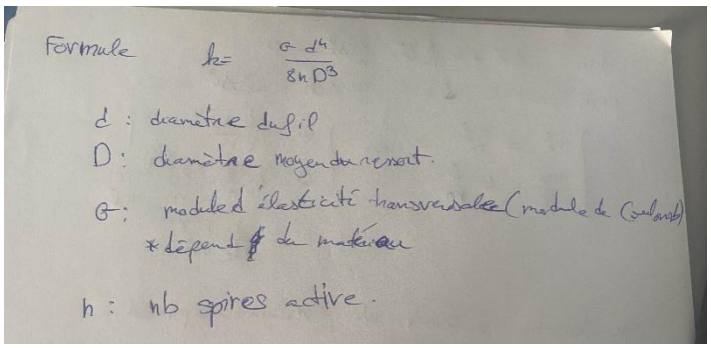
À l'aide théorème du moment cinétique, j'ai pu trouver des conditions pour avoir un moment nul pour mon solide « support de caméra/téléphone ». Pour une modélisation réaliste d'un mouvement à 1D, j'ai choisi de mettre deux ressorts sur chacun des côtés du support. En effet la présence de deux ressorts de chaque côté permet une meilleure stabilisation si un mouvement de rotation est initié.

Les conditions trouvées sur d'avoir une unique constante de raideur pour tous les ressorts et d'avoir une longueur à vide égale pour les deux ressorts du même côté du support (1 et 2 / 3 et 4).
Une étude dynamique du support nous mène à un système oscillateur amorti.

Les frottements étant liés à la constante de raideur.

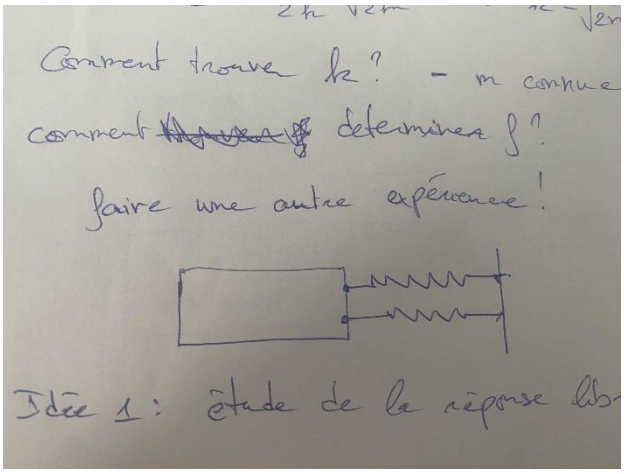
On peut déterminer les frottements existants pour choisir la constante de raideur idéal pour le système.
On veut avoir une réponse libre apériodique critique car elle permet d'avoir une réponse relativement rapide et sans oscillation (très peu d'oscillation en réalité avec les incertitudes de mesures et les simplifications) ce qui est un bon compromis entre vitesse du système et mouvement voulu pour revenir à la position d'équilibre.

De plus, cette formule trouvée dans des cours d'école d'ingénieur sur la résistance des



matériaux permet de choisir les dimensions des ressorts pour obtenir la constante de raideur que l'on souhaite et donc de les commander.

Pour déterminer les forces de frottements que le système subi, on peut s'aider d'une expérience annexe comme celle-ci



Les ressorts utilisés dans cette expérience étant de constante de raideur connue (soit déterminé à l'aide d'une autre expérience (masse suspendue + élongation -> on peut remonter à la constante de raideur) soit par la formule citée plus haut.

Ainsi on peut déterminer les forces de frottements que subie notre système et choisir, avec les exigences de dimension dû au slider, la constance de raideur idéale des ressorts pour le système de stabilisation passif.

Cette solution me paraissait correcte mais n'a pas été satisfaisant pour mon professeur encadrant car il pense la faible masse du système {support + téléphone/caméra} rendrait les frottements trop peu significatifs pour permettre d'avoir la réponse voulue à une perturbation.

J'aimerais vérifier ceci par le calcul.
.... Démarche :

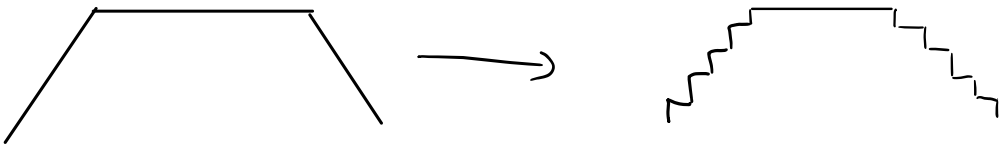
- A compléter

Idée 2 :

Modifier numériquement la consigne de vitesse (ne pouvant être qu'un échelon) pour en faire un trapèze.

Succession de plusieurs ordres à vitesse variable pour simuler une consigne à vitesse croissante, puis constante puis décroissante.

Discrétisation du trapèze à l'aide d'échelon



A voir comment réagit le slider à une série de consigne à différente vitesses.

Risque de encore plus avoir d'à-coups.

Modèle d’intelligence artificiel :

Puis j’essayerai d’entrainer un modèle d’IA sur une base de données que j’aurai crée